

Peer-to-peer file sharing (P2P) systems like Gnutella, KaZaA, and eDonkey/eMule, have become extremely popular in recent years, with the estimated user population in the millions. An academic research paper analyzed Gnutella and eMule protocols and found weaknesses in the protocol; many of the issues found in these networks are fundamental and probably common on other P2P networks. Users of file sharing networks, such as eMule and Gnutella, are subject to monitoring of their activity. Clients may be tracked by IP address, DNS name, software version they use, files they share, queries they initiate, and queries they answer to. Clients may also share their private files to the network without notice due to inappropriate settings.

Much is known about the network structure, routing schemes, performance load and fault tolerance of P2P systems in general. The eMule protocol does not provide much privacy to the users, even though it is a P2P protocol which is supposed to be decentralized.

The Gnutella and eMule protocols

The eMule protocol

eMule is one of the clients which implements the eDonkey network. The eMule protocol consists of more than 75 types of messages. When an eMule client connects to the network, it first gets a list of known eMule servers which can be obtained from the Internet. Despite the fact that there are millions of eMule clients, there are only small amount of servers. The client connects to a server with TCP connection. That stays open as long as the client is connected to the network. Upon connecting, the client sends a list of its shared files to the server. By this the server builds a database with the files that reside on this client. The server also returns a list of other known servers. The server returns an ID to the client, which is a unique client identifier within the system. The server can only generate query replies to clients which are directly connected to it. The download is done by dividing the file into parts and asking each client a part.

The Gnutella protocol

Gnutella protocol v0.4

In Gnutella protocol V0.4 all the nodes are identical, and every node may choose to connect to every other. The Gnutella protocol consist of 5 message types: query for file search. Query messages use a flooding mechanism, i.e. each node that receives a query forwards it on all of its adjacent graph node links. A node that receives a query and has the appropriate file replies with a query hit message. A hop count field in the header limits the message lifetime. Ping and Pong messages are used for detecting new nodes that can be linked to the actual file download performed by opening TCP connection and using the HTTP GET mechanism.

Gnutella protocol v0.6

Gnutella protocol V0.6 includes several modifications: A node has one of two operational modes: "leaf node" or "ultrapeer". Initially each node starts in a leaf node mode in which it can only connect to ultrapeers. The leaf nodes send query to an ultrapeer, the ultrapeer forwards the query and waits for the replies. When a node has enough bandwidth and uptime, the node may become an ultrapeer. Ultrapeers send periodically a request for their clients to send a list with the shared files they have. If a query arrives with a search string that matches one of the files in the leaves, the ultrapeer replies and pointing to the specific leaf.

Tracking initiators and responders

In version 0.4 of the Gnutella protocol, an ultrapeer which receives a message from a leaf node (message with hop count zero) knows for sure that the message was originated from that leaf node.

In version 0.6 of the protocol, If an ultrapeer receives a message from an ultrapeer with hop count zero then it knows that the message originated by the ultrapeer or by one of its leaves

(The average number of the leaves nodes that are connected to an ultrapeer is 200).

Tracking a single node

Many clients of Gnutella have an HTTP monitor feature. This feature allows sending information about the node to any node which supports an empty HTTP request, and receiving on response. Research shows that a simple crawler which is connected to Gnutella network can get from an initial entry point a list of IP addresses which are connected to that entry point. Then the crawler can continue to inquire for other IP addresses. An academic research performed the following experiment: At NYU, a regular Gnucleus software client that was connected to the Gnutella network as a leaf node, with distinctive listening TCP port 44121. At the Hebrew University of Jerusalem, a crawler ran looking for client listening with port 44121. In less than 15 minutes the crawler found the IP address of the Gnucleus client in NYU with the unique port.

IP address harvesting

If a user is connected to the Gnutella network within, say, the last 24 hours, that user's IP address can be easily harvested by hackers, since the HTTP monitoring feature can collect about 300,000 unique addresses within 10 hours.

Tracking nodes by GUID creation

A Globally unique identifier (GUID) is a 16 bytes field in the Gnutella message header, which uniquely identifies every Gnutella message. The protocol does not specify how to generate the GUID.

Gnucleus on Windows uses the Ethernet MAC address used as the GUID 6 lower bytes. Therefore, Windows clients reveal their MAC address when sending queries.

In the JTella 0.7 client software the GUID is created using the Java random number without an

initialization. Therefore, on each session, the client creates a sequence of queries with the same repeating IDs. Over time, a correlation between the user queries can be found.

Collecting miscellaneous information users

The monitoring facility of Gnutella reveals an abundance of precious information on its users. It is possible to collect the information about the software vendor and the version that the clients use. Other statistical information about the client is available as well: capacity, uptime, local files etc.

In Gnutella V0.6, information about client software can be collected (even if the client does not support HTTP monitoring). The information is found in the first two messages connection handshake.

Tracking users by partial information

Some Gnutella users have a small look-alike set, which makes it easier to track them by knowing this very partial information.

Tracking users by queries

An academic research team performed the following experiment: the team ran five Gnutella as ultrapeer (in order to listen to other nodes' queries). The team revealed about 6% of the queries.

Usage of hash functions

SHA-1 hashes refer to SHA-1 of files not search strings.

Half of the search queries are strings and half of them are the output of a hash function (SHA-1) applied on the string. Although the usage of hash function is intended to improve the privacy, an academic research showed that the query content can be exposed easily by a

dictionary attack: collaborators ultrapeers can gradually collect common search strings, calculate their hash value and store them into a dictionary. When a hashed query arrives, each collaborated ultrapeer can check matches with the dictionary and expose the original string accordingly.

Measures

A common countermeasure used is concealing a user's IP address when downloading or uploading content by using anonymous networks, such as I2P. There is also data encryption and the use of indirect connections (mix networks) to exchange data between peers.

Thus all traffic is anonymized and encrypted. Unfortunately, anonymity and safety come at the price of much lower speeds, and due to the nature of those networks being internal networks there currently still is less content. However, this will change, once there are more users.

See also

Gnutella2, a reworked network based on Gnutella

Bitzi, an open content file catalog integrated with some Gnutella clients

Torrent poisoning

References

Further reading

A Quantitative Analysis of the Gnutella Network Traffic - Zeinalipour-Yazti, Folias - 2002

Crawling Gnutella: Lessons Learned - Deschenes, Weber, Davison - 2004

Security Aspects of Napster and Gnutella Steven M. Bellovin 2001

Firewalls and Internet Security: Repelling the Wily Hacker, Second Edition

Daswani, Neil; Garcia-Molina, Hector. Query-Flood DoS Attacks in Gnutella

eMule Protocol Specification by Danny Bickson and Yoram Kulbak from HUJI.

External links

[eMule project Official website](#)

[eMule on SourceForge \(SourceForge\)](#) Contains archives of past versions of eMule

[List of allowed eMule-Mods](#)